Universitat Autònoma de Barcelona

Facultat de ciències

PRÀCTICA 2:
Zeros de funcions: Òrbites de Lyapunov

Grau en Matemàtiques

Guillem Gaitan Roca

2023/2024

Índex

1 Introducció i objectius **2**

2 Implementació del software **2**

2.1 Mètodes de la bissecció i Newton 2

2.2 Propagació d'una trajectòria 3

2.3 Càlcul de `x0` per a trobar una òrbita de Lyaponov 4

2.4 Orbites de Lyapunov addicionals 4

3 Manual de software **5**

3.1 Software de biblioteques 5

3.1.1 `zeros.c` 5

3.2 Software d'utilitats 6

3.2.1 `rtbp_lyap_csig.c` 6

3.2.2 `rtbp_lyap_ref.c` 7

3.2.3 `rtbp_sim.c` 7

3.2.4 `proves_bis_exp.c` 7

3.2.5 `proves_nwt_exp.c` 7

3.2.6 `cont_num.c` 8

4 Descripció de les proves realitzades **8**

4.1 Proves amb la funció $f(x) = e^x - 2$: 8

4.1.1 Bisecció 8

4.1.2 Newton 9

4.2 Determinació d'una òrbita de Lyapunov plana 10

4.3 Òrbites de Lyapunov addicionals 12

1 Introducció i objectius

El problema dels tres cossos consisteix en determinar les òrbites que descriuen tres masses que interaccionen entre elles. És un problema sense solució analítica i per resoldre'l s'han de considerar diverses aproximacions; per una banda els cossos més massius són masses puntuals en moviment circular uniforme l'un al voltant de l'altre, i el tercer es considera de massa negligible. A banda, per simplificar les equacions del moviment les òrbites han de ser coplanàries. L'objectiu d'aquesta pràctica és trobar òrbites de Lyapunov planes en un sistema format per la Terra, la Lluna i un satèl·lit, resolent el problema restringit de tres cossos.

Les òrbites de Lyapunov són trajectòries planes, tancades i estables al voltant d'un punt de Lagrange, que són punts on l'atracció gravitatòria generada per diferents cosso celestes s'anul·la. Les equacions de moviment que resoldrem són les següents:

$$\begin{cases} \ddot{x} - 2\dot{y} = \partial_x \Omega(x,y) \\ \ddot{y} + 2\dot{x} = \partial_y \Omega(x,y) \end{cases} \tag{1}$$

on hem fet un canvi d'unitats per tal que la distància entre la Lluna i al Terra sigui constant i igual a 1, i s'ha definit:

$$\begin{aligned} \Omega(x,y) &:= \frac{1}{2}(x^2+y^2) + \frac{1-\mu}{\rho_1} + \frac{\mu}{\rho_2} + \frac{1}{2}\mu(1-\mu) \\ \rho_1 &:= ((x-\mu)^2+y^2)^{1/2} \\ \rho_2 &:= ((x-\mu-1)^2+y^2)^{1/2} \end{aligned}$$

on $\mu = m_2/(m_1+m_2)$ és el paràmetre de masses. Finalment, trobarem les òrbites de Lyapunov per una certa *Constant de Jacobi*, que es defineix com $C(x,y,u,v) = -(u^2-v^2) + 2\Omega(x,y)$. En la pràctica s'implementaran els mètodes per trobar zeros de funcions de bisecció i Newton per determinar la x inicial tal que per una μ i C determinada trobem una òrbita de Lyapunov (Sempre considerarem $y_o = 0, v_o = 0$). Com a part complementària s'implementarà el mètode de continuació numèrica per determinar òrbites per altres valors de C.

2 Implementació del software

Expliquem el software separant les diferents fases de la implementació:

2.1 Mètodes de la bissecció i Newton

Per la primera part s'ha optat per implementar la rutina `bisecció()` utilitzant l'Algorisme 2.1 del guió, i la rutina `newton()` amb l'algorisme 2.2. Ambdues rutines estan detallades en el manual de software de la llibreria `zeros.c`. Els algorismes són els següents:

Algorisme 2.1 (mètode de la bisecció)
$a_1 := a$, $b_1 := b$, $I_1 := [a_1, b_1]$. $\forall i = 1, 2, 3, \ldots$

$p_i := \frac{a_i + b_i}{2}$

Si $f(a_i)f(p_i) \leq 0$

$a_{i+1} := a_i$, $b_{i+1} := p_i$

Si no

$a_{i+1} := p_i$, $b_{i+1} := b_i$

$I_{i+1} := [a_{i+1}, b_{i+1}]$

Algorisme 2.2 (mètode de Newton)
$\forall i = 0, 1, \ldots, \text{maxit} - 1$,

$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$

Si $|f(x_{i+1})| \leq \text{tol}_f$ y $|x_{i+1} - x_i| \leq \text{tol}_x$, tornem x_{i+1}, STOP.

Error (no convergència).

2.2 Propagació d'una trajectòria

El següent objectiu de la pràctica és propagar trajectòries per diferents x_0 amb la funció `proptraj()`, definida en la llibreria `rtbp.c`, per torbar-ne una que talli perpendicularment l'eix x i que per tant sigui tancada (u al punt de tall ha de ser 0). Les trajectòries es propaguen amb els següents paràmetres fixats: la constant de Jacobi $C = 3.196255396314419$, el paràmetre de masses Lluna-Terra $\mu = 1.215058560962404 \cdot 10^{-2}$.

Aquest apartat s'ha desenvolupat en la utilitat `rtbp_lyap_csig.c` i s'ha optat per fer un bucle `for` que per cada iteració genera un valor x_i, per $i = 1, \ldots, n$:

$$x_i = x_0 + i\frac{(x_f - x_0)}{n} \tag{2}$$

on x_0 és el valor més petit que pot prendre x_i i x_f el més gran. Amb aquest valor de x_i es propaga la trajectòria amb la rutina `proptraj()` i s'imprimeix per pantalla `xi, ui`, on u_i és la component x de la velocitat al punt de tall. La rutina `proptraj()` també genera un fitxer amb els valors calculats que ens permet representar gràficament la component x de la velocitat quan talla l'eix en funció de la posició inicial.

2.3 Càlcul de `x0` per a trobar una òrbita de Lyaponov

Finalment, s'ha de determinar la coordenada x de la posició inicial $(x_0, 0)$ per tal de trobar una òrbita de Lyaponov plana. Primer de tot s'ha definit una estructura `estructura = double c, int isgn` per facilitar l'escriptura dels següents passos. L'objectiu és adaptar la utilitat `ptoptraj()` per tal de poder-la implementar a les rutines `biseccio()` i `newton()` i trobar el zero de la funció, que és la x per la qual la velocitat u al punt de tall és zero.

S'han definit dues funcions, la primera és `lyap_bis_fdf()` que té el prototipus `double lyap_bis_fdf (double x, void *prm)`, com a arguments es passen un valor de `x` i l'estructura que s'ha definit abans, i propaga la trajectòria amb `proptraj()` en `x` retornant `u`. La segona és `lyap_nwt_fdf` que té el prototipus `void lyap_nwt_fdf (double x, double *v, double *dv, void *prm)`. Es passen com arguments la posició `x`, dos punters on es guardaran els valors de la velocitat `u` i `du`, i finalment l'estructura definida.

Dins del `main` es defineixen els paràmetres de l'estructura, al `double c` se li assigna el valor de la constant de Jacobi i al `int isgn` un valor `1` o `-1`, que indica el sentit cap on es propagarà la trajectòria. A continuació, amb la rutina `biseccio()` es troba una aproximació del zero de la funció `lyap_bis_fdf()`, i amb la rutina `newton()` es determina, amb una tolerància determinada, el zero de la funció `lyap_nwt_fdf`. Amb aquest zero després es podrà propagar una òrbita de Lyapunov plana i representar-la amb `gnuplot` .

2.4 Orbites de Lyapunov addicionals

Un cop s'ha trobat una òrbita de Lyapunov es pot automatitzar el procés per determinar-ne d'altres. En la utilitat `cont_num.c` s'utilitza el mètode de continuació numèrica sobre `c`, que consisteix en fer variar contínuament un paràmetre i estudiar com va evolucionant el sistema, per trobar altres òrbites. S'ha definit un `double l_step`, que és la variació que se li fa a `c` i per no fer el bucle infinit també es defineix un valor màxim de pas: `double l_max`. Es fa córrer un bucle `while` mentre `i*l_step < l_max`, dins d'aquest es fa una variació del paràmetre `c`: `c = c - l_step` (es resta ja que es busquen òrbites de més energia, i per tant de constant de Jacobi més petita). Amb el nou valor de `c` es crida la rutina `newton()` per a trobar el zero de la funció `lyap_nwt_fdf()`, és a dir, per trobar la posició inicial que correspon a l'orbita de la `c` determinada. Si la rutina no convergeix no retornarà res, i si s'arriba a un valor s'imprimeix per pantalla. Un cop es té la llista de posicions i constants de Jacobi, se'n seleccionen algunes i es representen amb `gnuplot`.

3 Manual de software

3.1 Software de biblioteques

3.1.1 `zeros.c`

Les dues rutines que hi ha en la biblioteca `zeros.c` són:

1. **La rutina** `biseccio()`:
   La rutina `biseccio()` utilitza el mètode de bisecció (Algorisme 2.1) per acotar l'interval en que es troba un zero d'una funció `f` i retorna el valor intermig de l'interval en la última iteració. Dins d'un bucle `while` s'aniran actualitzant els valors de `a` i `b` per anar acotant l'interval de la següent manera: primer es calcula `p=(a+b)/2`, s'avalua la funció de la que es busca el zero en el punt `a` i `p`, i es fa el producte de les dues funcions. Si el producte `f(a)*f(p)` $\leq$ `0`, la funció entre aquests dos punt passarà pel zero (ja que hi ha un canvi de signe), i es redefinirà el paràmetre `b = p`, si no es compleix aquesta condició s'assignarà `a = p`, i finalment es calcularà la longitud del nou interval, que, com que `p` és el punt mig de l'interval de la iteració prèvia, la nova longitud serà simplement `k/2`. Aquest bucle `while` continuarà mentre `k` $\geq$ `tol`. La funció retorna l'últim `p` que s'hagi calculat. Si `tol` és més gran o bé igual a l'interval inicial, la funció retorna el valor de `a`. També retorna els valors dels extrems de l'interval final en els punters `a` i `b`.
   Prototipus:
   `double biseccio(double *a, double *b, double tol, double (*f)(double,void*), coid *prm, int ixrr)`
   Rep:
   - `a, b`: valors dels extrems de l'interval inicial amb `a < b`.
   - `tol`: tolerància en `*x`. Quan s'assoleix `k` $\leq$ `tol`, s'atura el bucle `while`.
   - `f`: funció de la qual estem buscant un zero. Ha de tenir com a argument un `double` i opcionalment un `void`.
   - `prm`: argument de la funció `f`.
   - `ixrr`: índex de xerrera, si `ixrr = 1`, quan s'executi la rutina, s'aniran imprimint per pantalla els valors de `p` que es troben en cada iteració.

   Torna: el valor de `p` de la última iteració.

2. **La rutina** `newton()`:
   Aquesta rutina utilitza l'algorisme de Newton (Algorisme 2.2) per trobar zeros de funcions. Primer de tot s'avalua la funció i la seva derivada en `*x`, per tal d'inicialitzar els doubles `fx` i `dfx` respectivament. Dins d'un bucle `for` s'assigna el valor d'un double `xi = *x`, `*x` pren un nou valor amb la recurrència: `*x = *x - fx/(dfx)` i es torna a avaluar la funció amb el nou valor de `*x`. En el cas que tant la funció com la `*x` hagin arribat a la tolerància demanada, és a dir, que `fabs(fx) <= tolf` i `fabs(x-xi) <= tolx`, llavors el bucle s'aturarà i la rutina retornarà el nombre d'iterats que ha fet. Si s'arriba al nombre màxim

d'iterats i no es compleixen les condicions, llavors, la rutina retornarà `-1`.
Prototipus:
`int newton(double *x, double tolf, double tolx, int maxit, void (*fdf)(double, double*, souble*, void*), void *prm, int ixrr)`
Rep:

- `x`: valor inicial amb el qual començarà la recurrència.
- `tolf`: tolerància en el valor de la funció, quan `fx` $\leq$ `tolf`, el bucle `for` s'atura.
- `tolx`: tolerància en `*x`. Quan la diferència, en valor absolut, entre dos valors de `*x` de dues iteracions consecutives assoleix `xi - x*` $\leq$ `tolx`, s'atura el bucle `for`.
- `maxit`: nombre màxim d'iterats que farà la rutina abans no s'aturi.
- `fdf`: funció de la qual estem buscant un zero. Ha de tenir com a argument un `double`, dos `* double` i opcionalment un `void`. Ha de calcular el valor de la funció i la seva derivada per un valor `*x`.
- `prm`: argument de la funció `f`.
- `ixrr`: índex de xerrera, si `ixrr = 1`, quan s'executi la rutina, s'aniran imprimint per pantalla els valors de `x` que es troben en cada iteració.

Torna: el nombre d'iterats que ha fet o bé `-1` si el mètode no ha arribat a la tolerància demanada.

3.2 Software d'utilitats

3.2.1 `rtbp_lyap_csig.c`

La utilitat `rtbp_lyap_csig.c` imprimeix per pantalla els valors de la posició inicial calculada amb l'expressió (2) i la corresponent velocitat quan talla l'eix x:

$$x_i \qquad u_{i,f}$$

Per generar l'executable `rtbp_lyap_csig` s'ha d'introduir `make rtbp_lyap_csig`[1] a la consola. La utilitat respon a la crida:

```
./rtbp_lyap_csig c isgn xi xf nx
```

En el nostre cas en comptes d'imprimir els valors per pantalla es vol generar un fitxer u.txt que contingui les dades. Per fer-ho s'introdueix la comanda, amb uns valors d'exemple:

```
./rtbp_lyap_csig 3.196255396314419=c -1=isgn -0.830=xi -0.829=xf 100=nx > u.txt
```

[1] Les instruccions `make` utilitzen el fitxer `Makefile` per automatitzar la creació d'objectes i executables. Sempre que s'introdueixi una comanda `make`, aquesta estarà definida en el `Makefile`.

3.2.2 `rtbp_lyap_ref.c`

La utilitat `rtbp_lyap_ref.c` imprimeix per pantalla la x que s'obté després de fer bisecció i Newton. Amb la comanda `make rtbp_lyap_ref` es genera l'executable `make rtbp_lyap_ref` que respon a la crida:

```
./rtbp_lyap_ref c isgn xa xb tolbis tolfnwt maxitnwt ixrr
```

Un exemple d'una crida d'aquesta utilitat és:

```
./rtbp_lyap_ref 3.196255396314419=c -1=isgn -0.830=xa -0.829=xb 1e-4=tolbis
1e-12=tolfnwt 10=maxitnwt 1=ixrr
```

3.2.3 `rtbp_sim.c`

La utilitat `rtbp_sim.c` crida la rutina `protpraj()` per propagar una certa trajectòria sortint perpendicularment de l'eix x i fins a un cert nombre de talls `nt`. Es genera l'executable amb `make rtbp_sim`, i respon a la crida:

```
./rtbp_sim mu pmax
```

on `mu` és el paràmetre de masses i `pmax` el pas màxim d'integració numèrica. Per generar els punts de la trajectòria s'utilitza la comanda `echo` de la terminal, que imprimeix per standard output les dades de la propagació mitjançant la comanda:

```
echo x c isgn nt | ./rtbp_sim _=mu _=pmax > fitxer.txt
```

En aquest cas els valors es guarden en el fitxer `fitxer.txt` per poder representar l'orbita. Si en comptes d'escriure un valor s'escriu _, llavors pren com a valors els valors definits per defecte que són `mu` $= 1.215058560962404 \cdot 10^{-2}$ i `pmax` $= 0.01$.

3.2.4 `proves_bis_exp.c`

La utilitat `proves_bis_exp.c` fa passar la funció $f(x) = e^x - 2$ per la rutina `bisecció()` per determinar-ne el zero. L'executable es genera amb la comanda `make proves_bis_exp` i aquest respon a la comanda:

```
./proves_bis_exp a b tol irrx
```

3.2.5 `proves_nwt_exp.c`

La utilitat `proves_nwt_exp.c` fa passar la funció $f(x) = e^x - 2$ per la rutina `newton()` per determinar-ne el zero, el nombre d'iterats que ha fet la rutina i la compara amb el valor

$\ln(2) \approx 0.6931471805599453$. L'executable es genera amb la comanda `make proves_nwt_exp` i respon a la comanda:

```
./proves_bis_exp x tolf tolx maxit ixrr
```

3.2.6 `cont_num.c`

La utilitat `cont_num.c` imprimeix per pantalla parelles de valors `x0, c`, que corresponen a posicions incials i constants de Jacobi que determinen òrbites de Lyapunov planes. La utilitat utilitza el mètode de continuació numèrica sobre `c` per determinar-ne la `x0` corresponent. Per generar l'executable es fa la comanda `make cont_num` que respon a la crida:

```
./cont_num isgn x0 c0 l_max l_step tolfnwt maxitnwt ixrr
```

4 Descripció de les proves realitzades

4.1 Proves amb la funció $f(x) = e^x - 2$:

Per comprovar que les rutines `biseccio()` i `newton()` funcionen correctament, s'han fet proves amb una funció de la qual es coneix el zero amb precisió. Aquesta funció és $f(x) = e^x - 2$ que té un zero a $x = \ln 2$. Per comparar-ho amb els valors que s'obtenen, s'utilitza l'aproximació $\ln 2 \approx 0.6931471805599453$.

4.1.1 Bisecció

Per testejar la rutina s'han utilitzat els valors de la Taula 1, l'interval inicial és el mateix per totes les proves: `a = 0.5` i `b = 1`. El que es vol comprovar és que quan més iteracions, millor aproxima l'interval on es troba el zero de la funció i també es vol veure que realment s'atura quan arriba a la tolerància demanada. La primera part es veu directament de la taula 1, observem que valor de `k` $\leq$ `tol`, `k` és la longitud de l'interval de la última iteració, i a més, `x` pren el primer valor que compleix la condició, per tant la utilitat s'atura correctament. Observem que amb el mètode de la bisecció, més iterats no vol dir una millor aproximació del zero de la funció, es pot veure, per exemple, en que l'aproximació de la segona fila és millor que la de la tercera, però el que és important d'aquesta utilitat és que acoti cada cop millor l'interval on es troba el zero. Això és cert ja que per a cada iteració l'interval es divideix entre dos i es comprova que el zero de la funció es troba dins de cada interval.

Taula 1: Valors utilitzats i obtinguts de les proves de la rutina `biseccio()`.

`tol`	`i`	`x`	k	`[a,b]`
0.5	0	0.75	0.5	[0.5,1]
0.25	1	0.625	0.25	[0.5,0.75]

tol	i	x	k	[a,b]
0.15	2	0.6875	0.125	[0.625, 0.75]
0.125	2	0.6875	0.125	[0.625, 0.75]
0.0825	3	0.71875	0.0625	[0.6875,0.75]
0.0625	3	0.71875	0.0625	[0.6875,0.75]
0.03125	4	0.703125	0.03125	[0.6875,0.71875]
0.015625	5	0.6953125	0.015625	[0.6875,0.703125]

4.1.2 Newton

Per a testejar la rutina `newton()` s'han utilitzat els valors de la taula 2. En la primera, tercera i última prova s'arriba al valor exacte de referencia i per tant es comprova que el mètode dona bons resultats. No obstant, en el primer i tercer cas, l'algorisme només necessita 6 iteracions per trobar el valor mentre que en la última en necessita 15, això és degut que l'aproximació inicial de la última prova és dolenta. De fet, la quarta prova no convergeix ja que la x inicial dista molt del resultat inicial i en 10 iteracions no pot arribar al resultat amb la precisió demanada.

Taula 2: Valors utilitzats i obtinguts de les proves de la rutina `newton()`.

x0	tolf	tolx	maxit	i	x	x_i - ln(2)
2	10^{-8}	10^{-8}	10	6	0.6931471805599453	0
2	10^{-8}	1	10	5	0.6931471814512683	$8.91323 \cdot 10^{-10}$
2	1	10^{-8}	10	6	0.6931471805599453	0
10	10^{-12}	10^{-8}	10	-	NO CONVERGEIX	-
10	10^{-12}	10^{-8}	20	15	0.6931471805599453	0

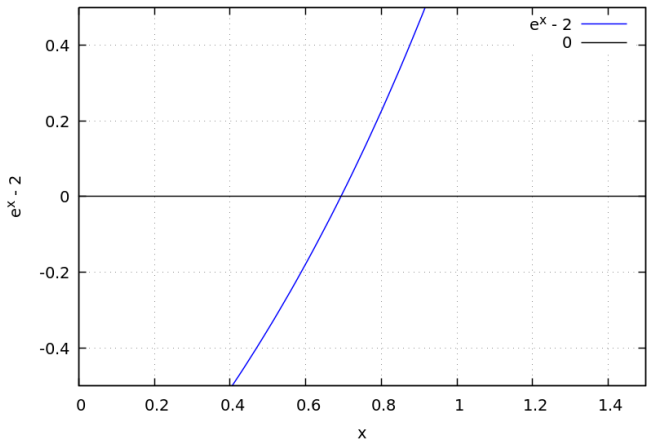

Figura 1: Representació de la funció $f(x) = e^x - 2$ en l'interval $[0, 1.5]$. S'ha representat de manera que l'escala dels dos eixos sigui la mateixa.

Ara només falta estudiar la diferencia entre la segona i la tercera prova. En els dos casos la estimació inicial és la mateixa, no obstant, la segona prova arriba a un resultat amb més precisió. Això és degut que la derivada de la funció $f(x) = e^x - 2$ en un entorn del zero és més gran que 1, com s'observa en la Figura 1. L'efecte que això provoca és que acotar l'interval de les x dona una millor aproximació que no pas buscar un valor petit de $f(x)$. Com que els resultats que proporciona la rutina concorden amb els teòrics, es pot concloure que `newton()` funciona correctament.

4.2 Determinació d'una òrbita de Lyapunov plana

L'objectiu principal de la pràctica és trobar una òrbita de Lyapunov plana, en aquest cas fixarem el valor de la constant de Jacobi en $C = 3.196255396314419$. El primer pas per a obtenir una òrbita de Lyapunov plana és determinar la posició inicial de la qual ha de partir. Per fer-ho, trobem una aproximació d'aquesta posició. Introduint la comanda:

```
./rtbp_lyap_csig 3.196255396314419=c -1=isgn -0.830=xi -0.829=xf 100=nx
> u.txt
```

es genera un fitxer `u.txt` amb les velocitats en el punt de tall amb l'eix x en funció de les posicions inicials. Després, amb `gnuplot` i introduint la comanda

```
plot 'u.txt' w l tit 'u en funció de x0', 0
```

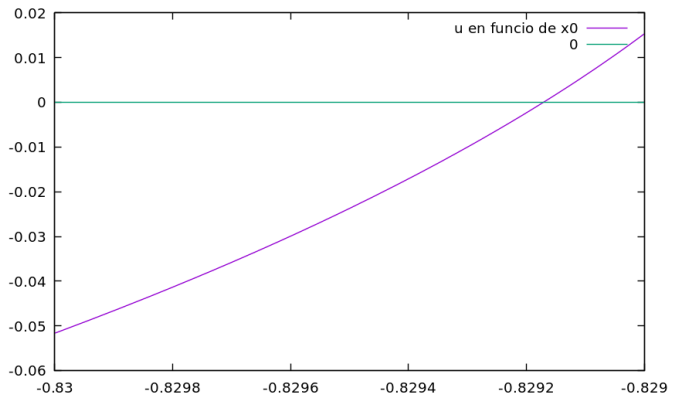

Figura 2: Representació de la velocitat al punt de tall en funció de la posició inicial.

es genera la figura 2. D'aquesta figura s'extreu que el zero que es busca està en un entorn de -0.8292. Per a representar l'òrbita, però, necessitem trobar el zero amb més precisió, cridem la utilitat `rtbp_lyap_ref.c` amb la comanda:

```
./rtbp_lyap_ref 3.196255396314419=c -1=isgn -0.830=xa -0.829=xb 1e-4=tolbis
1e-12=tolfnwt 10=maxitnwt 1=ixrr
```

amb la qual determinem $x = -0.8291710221854508$. Havent trobat la x inicial, ja podem propagar l'òrbita amb la comanda:

```
echo -0.8291710221854508 3.196255396314419 -1 6 | ./rtbp_sim _=mu _=pmax
> orbita_lyp.txt
```

on els paràmetres `mu` i `pmax`, corresponen a `mu` $= 1.215058560962404 \cdot 10^{-2}$ i `pmax` $= 0{,}01$. Finalment, representem l'orbita de la Figura 3 amb `gnuplot` amb la comanda `plot 'orbita_1.txt' u 2:3 w l tit 'orbita de Lyp'`.

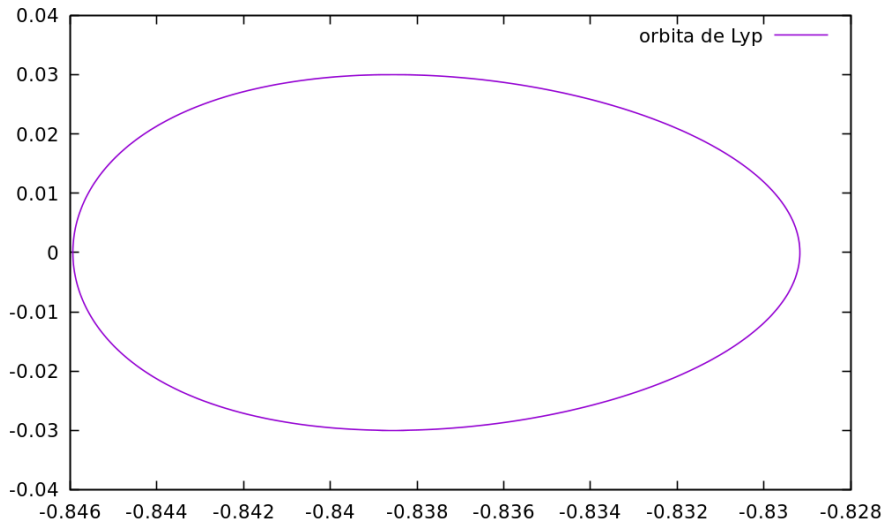

Figura 3: Representació de l'òrbita de Lyapunov tancada, propagada amb $x = -0.8291710221854508$.

Si volem representar l'òrbita amb la Terra i la Lluna, fem la crida:

```
plot 'orbita_lyp.txt' u 2:3 w l tit 'orbita de Lyp', 'lluna.txt' w cir tit 'Lluna',
>'terra.txt' w cir tit 'Terra'
```

En la representació de l'orbita només s'han representat fins a 6 talls amb l'eix horitzontal, observem que si es propaga la trajectòria amb més talls, l'òrbita ràpidament es desestabilitza i l'asteroide va a orbitar els cossos principals. Aquest efecte és degut que es troba la posició inicial amb un cert error que s'anirà propagant al llarg de la trajectòria a causa dels arrodoniments i truncaments i que fan que aquesta no sigui estable. En la Figura 4 s'ha representat la mateixa òrbita que en la Figura 3 però en aquest cas amb 20 talls amb l'eix. Per trobar òrbites que es mantinguessin estables més temps, necessitaríem determinar la posició inicial amb més preci-

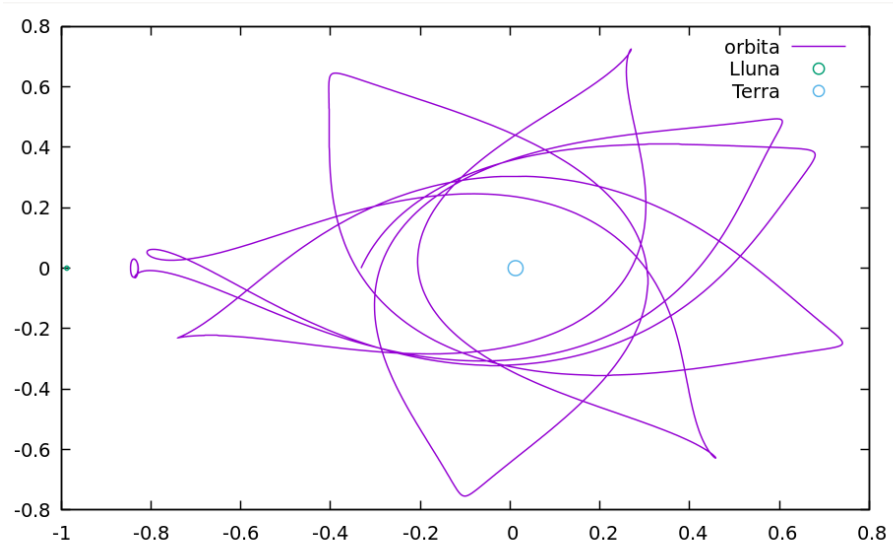

Figura 4: Representació de l'òrbita de Lyapunov tancada, propagada amb `x = -0.8291710221854508` i amb 20 talls.

sió, tot i això mai podrem determinar-lo exactament i per tant les òrbites en algun moment es desestabilitzaran.

4.3 Òrbites de Lyapunov addicionals

Per trobart més òrbites de Lyapunov fem ús de la utilitat `cont_num.c` per a generar parelles de valors de `x0` i `c`. Fem al comanda:

```
./cont_num -1=isgn -0.8291710221854508=x0 3.196255396314419=c0 0.3=l_max 1e-3=l_step
1e-12=tolfnwt 10=maxitnwt 0=ixrr
```

Taula 3: Valors utilitzats i obtinguts de les proves de la rutina `newton()`.

Òrbita	x0	c
1	-0.6051711048911562	2.899255396314452
2	-0.7710015411476108	3.015255396314439
3	-0.7954089974630379	3.069255396314433
4	-0.8101084674888922	3.069255396314433
5	-0.8148628499115801	3.157255396314423
6	-0.8291710221854508	3.196255396314419

Com que les òrbites de Lyapunov s'han de trobar en l'espai entre la Terra i la Lluna: `[-1,0]`, els paràmetres inicials han de correspondre a aquesta zona, per això utilitzem com a valors inicials

els valors de l'òrbita que s'ha trobat en l'apartat anterior. Fent proves s'ha vist que la màxima variació que es pot tenir de la constant de Jacobi és de l'ordre de 0.5 per això s'ha determinat una valor de pas de 0.001 i valor màxim de 0.3. Finalment, també s'ha fixat en 10 el nombre d'iterats màxims de la rutina de newton i l'índex de xerrera a 0 ja que no ens interessa veure el passos en cada iteració.

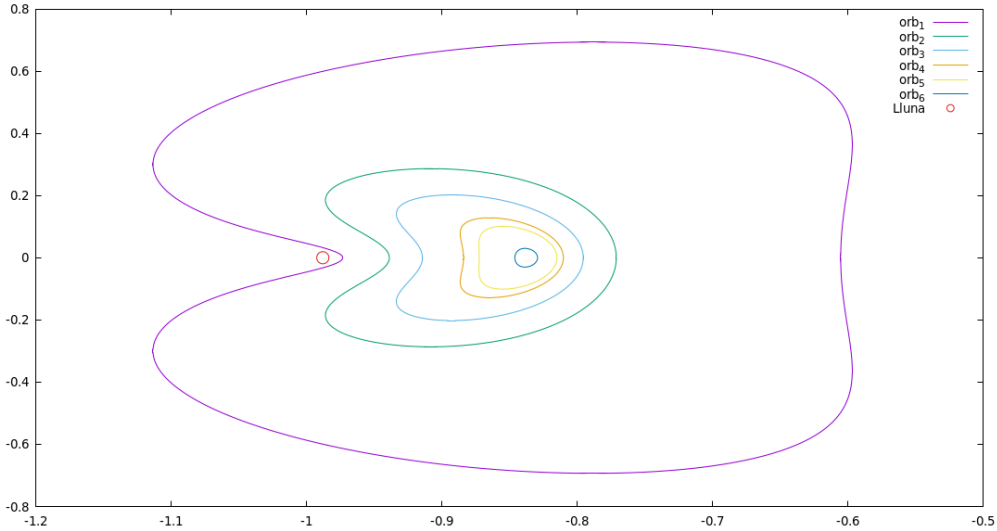

Figura 5: Representació de 6 òrbites de Lyapunov planes.

Els valors de la Taula 3 corresponen a alguns dels valors obtinguts, el 6 és l'òrbita trobada anteriorment. Aquests valors s'han representat, utilitzant `gnuplot`, en la Figura 5. Totes les trajectòries trobades orbiten al voltant del mateix punt de Lagrange. Observem que quan més petita la constant de Jacobi, i per tant, més gran l'energia, més a prop passa el satèl·lit de la Lluna. També es pot veure com cada cop les òrbites tracen trajectòries més deformades a causa dels efectes gravitatoris dels cossos principals.